ANDHAKANOON: AI-POWERED LEGAL CONTRACT ANALYZER

A Project Report Submitted in
Fulfillment of the Requirements
for the Degree of

Bachelor of Technology
In
Computer Science and Engineering (AI-ML)

Submitted By
Avni Shukla (2301661530016)
Deepshikha (2301661530025)
Shivam Agrahari (2301661530057)
Atul (2301661530015)

Under the supervision of
Mr. Sarvesh Patel



DR. AMBEDKAR INSTITUTE OF TECHNOLOGY FOR DIVYANGJAN
Affiliated to
DR. APJ ABDUL KALAM TECHNICAL UNIVERSITY

DECLARATION

We, the students of Computer Science and Engineering (AI-ML), Dr. Ambedkar Institute of Technology for Divyangjan, Kanpur, declare that our work entitled "AndhaKanoon: AI-Powered Legal Contract Analyzer" has been successfully completed under the guidance of Mr. Sarvesh Patel, Department of Computer Science and Engineering (AI-ML), Dr. Ambedkar Institute of Technology for Divyangjan, Kanpur. The dissertation work is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AI-ML) during the academic year 2024-25. Further, the matter embodied in this project report has not been submitted previously by anybody for the award of any degree of B.Tech to any University.

SIGNATURE OF STUDENTS -

Avni Shukla (2301661530016) ……..

Deepshikha (2301661530025) ……..

Shivam Agrahari (2301661530057) ……..

Atul (2301661530015) ……..

CERTIFICATE

This is to certify that the mini project entitled "AndhaKanoon: AI-Powered Legal Contract Analyzer" has been carried out by Avni Shukla, Deepshikha, Shivam Agrahari, Atul under my supervision in partial fulfillment for the degree of Bachelor of Technology (B.Tech) in Computer Science and Engineering (AI-ML) of Dr. Ambedkar Institute of Technology for Divyangjan under AKTU University during the academic year 2024-25.

MR. SARVESH PATEL

(Mini Project Coordinator)

Computer Science and Engineering (AI-ML)

MR. SRINATH DWIVEDI

HOD, Computer Science and Engineering (AI-ML)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success. The completion of this project, "AndhaKanoon: AI-Powered Legal Contract Analyzer", is attributable to the invaluable help and support we received from a number of people.

We are deeply grateful to our mentor and supervisor, Mr. Sarvesh Patel, for his constant guidance, encouragement, and technical direction throughout the development of this project. His insights into system design and problem-solving were instrumental in shaping the final outcome.

We would also like to express our sincere gratitude to Mr. Srinath Dwivedi, Head of Department, CSE (AI-ML), and to our Director, Prof. Rachna Asthana, for providing the infrastructure, resources, and encouragement necessary to complete this project.

Finally, we thank our classmates, families, and friends for their invaluable support and encouragement throughout this endeavour. The help received from each and every one, directly or indirectly, is duly acknowledged.

TEAM MEMBERS –

Avni Shukla (2301661530016)

Deepshikha (2301661530025)

Shivam Agrahari (2301661530057)

Atul (2301661530015)

ABSTRACT

Legal contracts are dense, formally worded documents, and their careful review is essential to protect the interests of the parties who sign them. In practice, however, thorough manual review is slow, expensive, and heavily dependent on the reviewer's expertise, which leaves individuals, freelancers, and small businesses in the Indian context particularly exposed to unfavourable or legally risky clauses. AndhaKanoon is an AI-powered legal contract analysis platform designed to close this gap. The system allows a user to upload a contract in PDF, DOCX, TXT, or image form; extracts and cleans its text; segments the document into individual clauses; and evaluates each clause using two complementary detection engines. The first is a deterministic, rule-based engine that matches clause text against a curated database of patterns drawn from the Indian Contract Act and related statutes. The second is a semantic engine that converts each clause into a dense vector embedding using Hugging Face transformer models and compares it against a ChromaDB vector store of known risky clause patterns, enabling the detection of risk even when it is expressed in paraphrased or unfamiliar language. The outputs of both engines are merged, deduplicated, and combined with a fairness-deviation module that compares the contract's terms against baseline fair-contract benchmarks for payment, termination, liability, and other key dimensions. A context-aware scoring module then produces a final risk score between 0 and 100, a categorical risk level, and a clause-by-clause, role-based explanation tailored to the perspective of a freelancer or a company. The system is built on a Next.js frontend and API layer, with SQLite for legal reference data, ChromaDB and Hugging Face for semantic search, and PostgreSQL through Prisma for persisted user accounts and saved analyses, with Clerk providing authentication. Users can view a highlighted, split-screen analysis, save it to their dashboard, download a formatted PDF report, or email the report to themselves or others. This report documents the motivation, related work, requirements, system architecture, folder-level design, implementation details, testing strategy, and results of the AndhaKanoon platform, and concludes with a discussion of its limitations and directions for future work, including multilingual support, expanded legal datasets, and a conversational legal assistant interface.

TABLE OF CONTENTS

Declaration ii
Certificate iii
Acknowledgement iv
Abstract v
List of Figures vii
List of Tables viii
Chapter 1: Introduction 1
1.1 Background and Motivation 1
1.2 Problem Statement 2
1.3 Objectives of the Project 3
1.4 Scope of the Project 3
1.5 Organization of the Report 4
Chapter 2: Literature Review 5
2.1 Rule-Based and Keyword Approaches 5
2.2 Transformer-Based Legal NLP 5
2.3 Comparable Systems 6
2.4 Research Gap Addressed by AndhaKanoon 7
Chapter 3: System Analysis and Requirements 8
3.1 Functional Requirements 8
3.2 Non-Functional Requirements 9
3.3 Feasibility Study 10
3.4 Hardware and Software Requirements 11
Chapter 4: System Architecture and Design 12
4.1 High-Level Architecture 12
4.2 Project Folder Structure 13
4.3 Data Stores 15
4.4 API Endpoints 16
Chapter 5: Technology Stack 17
Chapter 6: Detailed Working and Data Flow 19
6.1 Step-by-Step Analysis Pipeline 19
6.2 End-to-End Data Flow 23
6.3 Result Rendering and User Actions 24
Chapter 7: Implementation 25
Chapter 8: Testing 27
Chapter 9: Results and Discussion 28

Chapter 10: Ethical Considerations 29
Chapter 11: Conclusion and Future Scope 30
References 31

LIST OF FIGURES

Fig. 1: High-Level System Architecture of AndhaKanoon
Fig. 2: Functional Workflow of the Contract Analysis Pipeline
Fig. 3: Clause Parsing Strategy
Fig. 4: Dual-Engine Risk Detection (Keyword + Semantic)
Fig. 5: Context-Aware Risk Scoring Model
Fig. 6: Split-Screen Result Page Layout
Fig. 7: End-to-End Data Flow Pipeline

LIST OF TABLES

Table 3.1: Functional Requirements Summary
Table 3.2: Hardware and Software Requirements
Table 4.1: Root-Level Folder Responsibilities
Table 4.2: API Endpoints and Roles
Table 5.1: Technology Stack Summary
Table 9.1: Evaluation Metrics

Chapter 1: Introduction

1.1 Background and Motivation

Contracts govern almost every significant personal and professional relationship, from freelance service agreements to employment offers, non-disclosure agreements, and vendor arrangements. Yet the language in which contracts are written is dense, formal, and often intentionally ambiguous, which makes it difficult for a non-lawyer to identify clauses that create risk, impose one-sided obligations, or deviate from what is normally considered fair. This problem is especially acute in India, where freelancers, small business owners, and first-time employees frequently sign contracts without legal counsel, either due to cost, time pressure, or simple unfamiliarity with the relevant provisions of the Indian Contract Act, 1872, and related legislation.

Recent advances in Natural Language Processing (NLP), and in particular transformer-based language models, have made it possible to build systems that read and interpret legal text with a level of contextual understanding that keyword-matching tools cannot achieve. At the same time, purely model-driven approaches can be unpredictable and difficult to justify to an end user who needs to trust the result. AndhaKanoon was conceived to combine the reliability of deterministic, rule-based legal pattern matching with the contextual power of semantic embedding search, so that the system can catch both clauses that use known risky legal language and clauses that convey the same risk in different words.

1.2 Problem Statement

Given a contract document in a common file format, there is no accessible, automated system tailored to the Indian legal context that can (a) reliably extract and segment the contract into clauses, (b) identify clauses that are legally risky or that deviate from fair market practice, (c) explain those risks in plain, role-appropriate language for both a freelancer and a company, and (d) present the result in an interactive, exportable format, without requiring the user to consult a lawyer for an initial review. AndhaKanoon addresses this gap by building an end-to-end AI-assisted contract analysis pipeline.

1.3 Objectives of the Project

1. To design a system that accepts contracts in PDF, DOCX, TXT, and scanned image formats and extracts clean, structured text from each.

2. To reliably split extracted text into individual clauses using a multi-strategy parsing approach.
3. To detect legally risky clauses using a rule-based engine grounded in Indian Contract Act sections and patterns.
4. To detect paraphrased or semantically risky clauses using embedding-based similarity search, so that risk is not missed simply because a clause avoids exact keyword matches.
5. To compare contract terms against fair-contract baselines and surface deviations in payment, termination, liability, and other key terms.
6. To compute a context-aware, explainable risk score (0–100) that accounts for contract type, industry, value, and duration.
7. To generate plain-language, role-based explanations (freelancer perspective and company perspective) for every flagged clause.
8. To provide a usable interface for reviewing, saving, exporting as PDF, and emailing the analysis.

1.4 Scope of the Project

The current scope of AndhaKanoon covers single-document contract analysis for common Indian contract types such as freelance agreements, service agreements, NDAs, and employment contracts, in English. The system supports document upload, automated extraction, dual-engine risk detection, fairness deviation checking, context-aware scoring, explanation generation, dashboard persistence for signed-in users, PDF export, and email delivery of reports. Contract drafting and template generation, contract comparison, and digital signature workflows are present as supporting modules within the same codebase but are considered secondary to the core analysis pipeline that is the focus of this report. Multilingual analysis and real-time collaborative review are treated as future work rather than part of the current scope.

1.5 Organization of the Report

Chapter 2 reviews related work in legal document analysis and situates AndhaKanoon relative to existing rule-based and transformer-based systems. Chapter 3 presents the functional and non-functional requirements and the feasibility study. Chapter 4 describes the system architecture, folder structure, and data stores. Chapter 5 details the technology stack. Chapter 6 walks through the complete working and data flow of the application, step by step. Chapter

7 discusses implementation details, Chapter 8 covers the testing strategy, Chapter 9 presents results and discussion, Chapter 10 addresses ethical considerations, and Chapter 11 concludes the report with a summary of contributions and directions for future work.

Chapter 2: Literature Review

2.1 Rule-Based and Keyword Approaches

Early work in legal document automation relied primarily on rule-based systems and keyword or regular-expression extraction. These systems match a document's text against curated lists of legal terms, clause headings, or statutory phrases. Their principal advantage is predictability: the reasons behind a flag are fully transparent and directly traceable to a specific rule or pattern, which is valuable in a legal context where explainability matters. Their key weakness is brittleness — a clause that expresses the same risk using different wording, synonyms, or a restructured sentence can slip past a purely keyword-based system undetected.

2.2 Transformer-Based Legal NLP

With the emergence of transformer architectures, models such as BERT and RoBERTa were fine-tuned on legal corpora to produce domain-specific variants such as LegalBERT, which improved clause classification and contextual understanding of legal text. These models can capture semantic meaning beyond exact keyword overlap, which allows a system to recognize a paraphrased risky clause. However, transformer-based classification and generation models are comparatively opaque: it is harder to point to the precise textual or statutory basis for a decision, and pure model output can be inconsistent across similar inputs without careful calibration.

2.3 Comparable Systems

Existing tools such as LEXNLP and ContractNLI provide clause extraction or entailment-based risk identification, and academic prototypes such as ContractLens combine transformer-based summarization (using models like BART Large CNN) with clause classification to produce role-based, plain-English summaries for lawyers, clients, and students through a Streamlit interface. These systems demonstrate the value of clause-level analysis and role-based communication, but, as the ContractLens study itself notes, most existing tools work in isolation, focusing on either summarization or risk identification rather than an integrated, end-to-end pipeline that also grounds its findings in a specific statutory context, such as the Indian Contract Act.

2.4 Research Gap Addressed by AndhaKanoon

AndhaKanoon is designed to combine the transparency of a deterministic, statute-grounded rule engine with the coverage of a semantic embedding search, rather than choosing one approach over the other. Every clause is evaluated by both engines, and the match source (keyword, semantic, or both) is retained and shown to the user, which preserves explainability even where the semantic engine contributes additional coverage. In addition, AndhaKanoon layers a fairness-deviation module on top of clause-level risk detection, comparing the contract as a whole against baseline fair-contract terms for dimensions such as payment, termination, and liability — a level of holistic, contract-wide analysis that is not typically emphasized in clause-only extraction tools. Finally, the system is explicitly built around the Indian Contract Act and Indian market context, which is underserved relative to the largely US- and EU-focused legal NLP literature and tooling.

Chapter 3: System Analysis and Requirements

3.1 Functional Requirements

Table 3.1 summarizes the core functional requirements identified for the system, derived directly from the intended user workflow of uploading a contract and receiving an actionable risk analysis.

Requirement	Description
Document Upload	Accept PDF, DOCX, TXT, and image contract files with optional context (contract type, industry, value, duration, user experience).
Input Validation	Validate file existence, MIME type, and maximum size before processing.
Text Extraction	Convert uploaded documents into clean, machine-readable text using format-specific extractors and OCR for images.
Clause Parsing	Segment extracted text into individual clauses using numbered-clause, section-heading, and paragraph-fallback strategies.
Rule-Based Risk Detection	Match clauses against a database of Indian Contract Act sections and legal patterns, with fallback patterns if the database is unavailable.
Semantic Risk Detection	Convert clauses into embeddings and compare them against a vector store of risky patterns to detect paraphrased risk.
Result Merging	Merge and deduplicate keyword and semantic matches, tagging the source of each match.
Fairness Deviation Check	Compare contract terms to fair-contract baselines and produce a list of deviations with AI-generated summaries.
Context-Aware Scoring	Compute a final risk score (0–100), risk level, and score breakdown using violation severity plus contract type, industry, value, and duration modifiers.
Explanation Generation	Generate role-based (freelancer / company) plain-language explanations for each risky clause, with text positions for highlighting.
Result Persistence and Export	Allow signed-in users to save analyses, download a PDF report, and email the report with an optional attachment.

Table 3.1: Functional Requirements Summary

3.2 Non-Functional Requirements

- Performance: contract analysis, from upload to result, should complete within a time frame acceptable for interactive use, even for moderately long contracts.
- Reliability: if the semantic search infrastructure (Hugging Face or ChromaDB) is temporarily unavailable, the system must degrade gracefully to keyword-only detection rather than failing outright.

- Security and Privacy: uploaded documents, being sensitive legal material, must be processed securely and not retained beyond what is required for the analysis and any explicit save action taken by the user.
- Usability: the split-screen result view, with highlighted clauses on one side and the analysis panel on the other, should let a non-expert user understand the findings without additional training.
- Scalability: the API layer should handle multiple concurrent analysis requests without significant degradation in response time.
- Maintainability: the modular separation between extraction, parsing, rule-based detection, semantic detection, scoring, and explanation generation should allow each module to be updated independently.

3.3 Feasibility Study

3.3.1 Technical Feasibility

All core components — Next.js for the web application, SQLite for structured legal reference data, Hugging Face and ChromaDB for semantic search, and PostgreSQL with Prisma for persisted application data — are mature, well-documented, and freely available technologies, making the project technically feasible within a college mini-project timeframe and typical development hardware.

3.3.2 Economic Feasibility

The technology stack relies primarily on open-source libraries and frameworks. Where hosted services are used (for example, managed PostgreSQL or authentication through Clerk), free or low-cost development tiers are sufficient for a prototype of this scale, keeping the project economically feasible for an academic setting.

3.3.3 Operational Feasibility

The target users — freelancers, small business owners, employees, and students — are already accustomed to uploading documents to web applications and reading dashboard-style summaries, so the operational learning curve for the proposed interface is expected to be low.

3.4 Hardware and Software Requirements

Category	Requirement
Processor	Intel i5 / AMD Ryzen 5 or higher (development machine)
RAM	8 GB minimum, 16 GB recommended for embedding generation

Category	Requirement
Storage	Minimum 5 GB free space for dependencies, models, and vector store
Operating System	Windows / Linux / macOS
Runtime	Node.js (Next.js application and API routes)
Database	SQLite (legal reference data), PostgreSQL (user and saved-analysis data)
AI / NLP Services	Hugging Face embedding models, ChromaDB vector database
Authentication	Clerk
Browser	Any modern browser (Chrome, Firefox, Edge) for the frontend

Table 3.2: Hardware and Software Requirements

Chapter 4: System Architecture and Design

4.1 High-Level Architecture

AndhaKanoon follows a layered architecture in which a Next.js frontend communicates with Next.js API route handlers, which in turn coordinate a set of backend services responsible for extraction, parsing, rule-based validation, semantic validation, fairness checking, scoring, and explanation generation. Structured legal reference data (Indian Contract Act sections, clause patterns, and fair-contract baselines) is stored in SQLite for fast, deterministic lookups, while a Hugging Face embedding model and a ChromaDB vector store together provide semantic similarity search over legal clause patterns. User accounts and saved analyses are persisted in PostgreSQL through Prisma, and Clerk manages user authentication and identity.

Fig. 1: High-Level System Architecture of AndhaKanoon

This separation of concerns means that the deterministic rule engine can operate independently of the semantic engine — if the embedding service or vector database becomes temporarily unavailable, the system continues to function using keyword-based detection alone, which keeps the platform resilient in production.

4.2 Project Folder Structure

The codebase is organized so that UI, business logic, and data access are clearly separated. Table 4.1 summarizes the responsibility of each root-level folder.

Folder	Responsibility
app/	Main Next.js application: UI pages, layouts, route handlers, and server actions.
components/	Reusable React UI components — analysis widgets, contract viewers, upload UI, and shared design-system elements.
lib/	Core business logic and backend utilities: database access, analysis services, scoring logic, validation, and embeddings.
prisma/	Prisma schema and model definitions for user accounts and saved analysis history.
public/	Static assets served directly by the browser, including sample contracts.
data/	Local data files, mainly contract and law reference data.
scripts/	Standalone scripts for setup and batch processing, including embedding generation for semantic search.
report_summary/	Output area for generated report/summary files.

Table 4.1: Root-Level Folder Responsibilities

4.2.1 app/ Breakdown

- app/api/ — backend route handlers that perform analysis, PDF generation, report sending, and health checks.
- app/dashboard/ — user dashboard showing saved analyses and history.
- app/result/ — analysis results page displaying risk score, risky clauses, and deviations.
- app/compare/ — page for comparing contracts side by side.
- app/create-contract/ — page for AI-assisted contract drafting.
- app/report/ — printable / shareable report presentation page.
- app/sign-in/, app/sign-up/, app/onboarding/ — authentication and onboarding flows handled through Clerk.
- app/page.tsx — home page where the user starts the upload and analysis workflow.

4.2.2 components/ Breakdown

- components/analysis/ — score display, clause cards, and deviation highlights used in the results panel.
- components/contract/ — components for viewing and interacting with contract text.
- components/upload/ — file selection and submission UI.
- components/email/ — UI for composing and sending report emails.
- components/ui/ — base building blocks such as buttons, badges, cards, dialogs, and progress bars.

4.2.3 lib/ Breakdown

- lib/db/ — SQLite and ChromaDB access layer, including schema, seed data, and query helpers.
- lib/services/ — core analysis logic: extraction, parsing, law validation, semantic search, scoring, and explanations.
- lib/types/ — shared TypeScript type definitions.
- lib/utils/ — shared utility functions for file handling, vectors, and general helpers.

4.3 Data Stores

SQLite (accessed via better-sqlite3) holds act_sections (Indian Contract Act sections), clause_patterns (keyword and regex legal patterns), fair_contract_baseline (benchmark terms for deviation checks), contract_analysis_context (analysis telemetry), and supporting tables for

contracts, templates, signatures, and the clause library used by the drafting features. ChromaDB holds a vector collection of legal clause patterns used for semantic matching, while Hugging Face serves as the embedding provider that converts legal text into the vectors ChromaDB compares against. PostgreSQL, accessed through Prisma, stores the User and SavedAnalysis models that back the dashboard and analysis history. Finally, the browser's localStorage temporarily holds the latest analysis payload so the result page can render immediately after an upload without an additional round trip.

4.4 API Endpoints

Endpoint	Role
POST /api/analyze	Runs the full contract analysis pipeline: extraction, parsing, risk detection, scoring, and explanation.
GET /api/health	Reports application, database, and service health and statistics.
GET / POST / PUT / DELETE /api/contracts	Contract CRUD operations, templates, and clause data.
POST /api/generate-contract	AI-assisted contract draft generation and enhancement.
POST /api/generate-pdf	Generates a PDF version of the analysis report.
POST /api/export-pdf	Generates a PDF from contract content for drafting workflows.
POST /api/send-report	Emails the analysis report, optionally with a PDF attachment.
GET /api/laws	Returns Indian legal sections and reference data.
GET / POST / DELETE /api/signatures	Manages digital signatures used in contract generation.

Table 4.2: API Endpoints and Roles

Chapter 5: Technology Stack

AndhaKanoon is built on a modern JavaScript/TypeScript stack centered on Next.js, which allows the frontend pages and backend API routes to live in a single, cohesive application. Table 5.1 summarizes the primary technologies used across the platform and the role each one plays.

Layer	Technology
Frontend	Next.js (App Router), React components
Backend / API	Next.js route handlers (app/api/*)
Legal Reference Database	SQLite via better-sqlite3
Semantic Embeddings	Hugging Face Transformers
Vector Search	ChromaDB (nearest-neighbor similarity search)
Application Database	PostgreSQL
ORM	Prisma
Authentication	Clerk
Text Extraction	pdf-parse (PDF), mammoth (DOCX), Tesseract OCR (images)
Clause Parsing	Custom multi-strategy parser (numbered clauses, section headings, paragraph fallback)
Client-Side Caching	Browser localStorage for immediate result rendering

Table 5.1: Technology Stack Summary

The choice of SQLite for legal reference data keeps lookups of Indian Contract Act sections and clause patterns fast and deterministic without the operational overhead of a hosted database, while PostgreSQL through Prisma provides the relational integrity and multi-user concurrency needed for user accounts and saved analyses. Hugging Face and ChromaDB together form the semantic layer: Hugging Face supplies pretrained embedding models capable of representing legal text as dense vectors, and ChromaDB indexes those vectors for efficient nearest-neighbor search, which is what allows the system to recognize risky language even when it does not match any predefined keyword pattern.

Chapter 6: Detailed Working and Data Flow

6.1 Step-by-Step Analysis Pipeline

Step A — Upload and Request Creation

The user begins on the home page, where they select a contract file and a language. The frontend packages the file into a FormData object and sends it in a POST request to /api/analyze. Optional contextual fields — contract type, industry, contract value, duration in months, and user experience level — can be attached to the same request, since these values later feed into the context-aware scoring stage.

Step B — Input Validation

The API layer validates that a file is present, checks its MIME type against the list of allowed formats (PDF, DOCX, TXT, and common image formats), and enforces a maximum file size, rejecting the request early if any check fails.

Step C — Text Extraction

Depending on the file type, the system selects the appropriate extractor: pdf-parse for PDF files, mammoth for DOCX files, a direct read for plain text files, and Tesseract OCR for images. Every extractor returns both the extracted text and metadata such as character count, page count, and word count, which are used later for reporting and for informing the scoring model.

Step D — Clause Parsing

The extracted text is split into individual clauses using a multi-strategy parser. The parser first attempts to detect explicitly numbered clauses; if that pattern does not match well, it falls back to detecting section headings; and if neither pattern is found, it falls back further to paragraph-level segmentation. The output is a structured array of clause objects, each carrying an identifier, the clause text, and its position within the document.

Fig. 3: Clause Parsing Strategy

Step E — Legal Risk Detection (Two Engines)

Each clause passes through two independent detection engines that run in parallel. The keyword engine, validateAgainstIndianLaw, matches clause text against the clause_patterns and act_sections tables, using a set of fallback patterns if the database is temporarily unavailable. The semantic engine, validateSemantic, converts each clause into an embedding using the Hugging Face model, queries ChromaDB for the most similar pre-stored risky pattern

vectors, and applies threshold filtering along with a safe-clause filter to reduce false positives. If the semantic infrastructure is unavailable for any reason, the pipeline continues using keyword detection alone rather than failing the request.

Fig. 4: Dual-Engine Risk Detection (Keyword + Semantic)

Step F — Merge and Deduplicate

The results of the keyword and semantic engines are merged into a single list of violations. Matches that refer to the same clause are deduplicated, and each surviving match is tagged with its source — keyword, semantic, or both — which preserves transparency about why a clause was flagged.

Step G — Fairness Deviation Analysis

Independently of clause-level risk detection, the contract as a whole is compared against fair_contract_baseline benchmark terms covering dimensions such as payment terms, termination conditions, and liability. Any deviation from these benchmarks is recorded, and an AI-generated summary is produced for each deviation so that the user can quickly understand what changed relative to a typical fair contract, without reading the full legal comparison themselves.

Step H — Context-Aware Scoring

An enhanced scoring module computes the final risk score by starting from a base score derived from the severity of detected violations, then applying modifiers based on contract type and industry, and further adjusting for contract value and duration. The output includes the final numeric score (0–100), a categorical risk level, a human-readable score explanation, and a structured breakdown of how each factor contributed to the final number.

Fig. 5: Context-Aware Risk Scoring Model

Step I — AI Explanation Generation

For every clause flagged as risky, the system generates two role-based explanations — one written from the perspective of a freelancer and one from the perspective of a company — so that either party to the contract can understand the practical implication of the clause in language relevant to their position. The start and end text positions of each clause are also computed at this stage so that the frontend contract viewer can highlight the exact passage being discussed.

Step J — Response and Persistence

The API assembles a single structured JSON response containing the document metadata and extracted text, the overall analysis and score breakdown, the risky clauses with their legal references and explanations, the fairness deviations, and a standard disclaimer. In parallel, the request's analysis context is asynchronously written to the SQLite contract_analysis_context table for telemetry and future model improvement, without blocking the response to the user.

Step K — Result Rendering and User Actions

On the frontend, the JSON response is stored in the browser's localStorage and the application navigates to the /result page. This page presents a split-screen view: the left panel is the ContractViewer, which renders the original document with risky clauses highlighted inline, and the right panel is the AnalysisPanel, which shows the risk score, the list of violations, and the fairness deviations. From this page, a signed-in user can save the analysis to their Prisma-backed dashboard, download a colour-coded PDF report, or send the report by email with an optional PDF attachment.

Fig. 6: Split-Screen Result Page Layout

6.2 End-to-End Data Flow

The complete pipeline, summarized as a single flow, proceeds as follows: the uploaded file becomes a FormData request to /api/analyze; the API produces extracted text and metadata; the text is parsed into clauses; the clauses are evaluated by both the keyword and semantic validators; the resulting violations are merged; the contract is checked for fairness deviations and AI summaries are generated; a context-aware risk score is computed; clause explanations and highlighting indices are generated; the full result is returned as JSON; the frontend stores it in localStorage and renders the /result page; and, optionally, the user saves the analysis to the database, generates a PDF, or emails the report.

Fig. 7: End-to-End Data Flow Pipeline

6.3 Result Rendering and User Actions

Beyond viewing the split-screen result, the platform supports three concrete follow-up actions. Saving an analysis writes a SavedAnalysis record linked to the authenticated user's account in PostgreSQL, making the result available later from the dashboard. Downloading a report calls /api/generate-pdf to produce a colour-coded PDF that mirrors the on-screen risk categorization, suitable for printing or forwarding to a counterparty. Emailing a report calls /api/send-report,

which sends the analysis, optionally including the generated PDF as an attachment, directly from the platform.

Chapter 7: Implementation

7.1 Development Approach

The project was implemented incrementally, starting with the text extraction and clause parsing pipeline, followed by the deterministic keyword-matching engine against the seeded SQLite legal database, and then the semantic engine once the Hugging Face embedding pipeline and ChromaDB collection were prepared using the scripts in scripts/generateEmbeddings.ts and scripts/generateEmbeddings.main.ts. The fairness deviation module and context-aware scorer were layered on top once both detection engines were producing reliable output, followed by the explanation generation step and, finally, the result UI, PDF export, and email delivery features.

7.2 Rule-Based Engine Implementation

The rule-based engine, validateAgainstIndianLaw, queries the clause_patterns table for regular-expression and keyword patterns linked to specific act_sections records. Each match returns the matched clause, the associated legal section, and a severity classification. If the database connection or pattern set is unavailable at runtime, the engine falls back to a hard-coded set of common risk patterns so that basic detection continues to function.

7.3 Semantic Engine Implementation

The semantic engine, validateSemantic, first converts each parsed clause into a dense embedding vector using a Hugging Face model. This vector is compared against the pre-computed embeddings of known risky clause patterns stored in ChromaDB, using nearest-neighbor similarity search. A similarity threshold and a safe-clause filter are applied to the raw nearest-neighbor results to suppress low-confidence or clearly benign matches, which reduces false positives relative to using raw similarity scores alone.

7.4 Scoring Module Implementation

The context-aware scorer begins with a base score derived from the number and severity of detected violations (both keyword and semantic), then applies weighted modifiers for contract type and industry, and further adjusts the result based on the declared contract value and duration in months, on the reasoning that a higher-value or longer-duration contract with the same violation should generally be treated as higher risk. The final output includes the numeric

score, a mapped risk level (for example, low, moderate, high), a plain-language explanation, and a breakdown object listing each contributing factor.

7.5 Explanation Generation

For each flagged clause, the explanation module produces two short, role-specific explanations — one addressed to a freelancer and one addressed to a company — so that the same clause can be understood from either side of the contract. Clause start and end character positions in the original extracted text are also computed here, which the frontend uses to highlight the exact clause in the ContractViewer component.

7.6 Frontend Implementation

The frontend is implemented with Next.js App Router pages and React components. The upload flow lives on the home page (app/page.tsx), and the result page (app/result/) renders the split-screen ContractViewer and AnalysisPanel components once the analysis JSON is retrieved from localStorage. Reusable UI primitives — buttons, cards, badges, dialogs, and progress indicators — are implemented once in components/ui/ and reused across the analysis, contract, upload, and email component groups, keeping the interface visually consistent.

7.7 Persistence and Authentication

Clerk manages user sign-in, sign-up, and onboarding, and every save, dashboard, or history request is scoped to the authenticated user's identity. Prisma provides a typed data-access layer over PostgreSQL for the User and SavedAnalysis models, and the Prisma schema (prisma/schema.prisma) defines the relationship between a user and their saved analyses.

Chapter 8: Testing

Because AndhaKanoon combines a deterministic rule engine with a probabilistic semantic engine, testing was designed to validate each engine independently as well as their combined behaviour, and to confirm that the system degrades gracefully rather than failing outright when a dependent service is unavailable. Table 8.1 summarizes the testing categories applied.

Test Category	Description
Unit Testing	Individual functions in lib/services (extraction, parsing, scoring) tested in isolation with representative sample inputs.
Integration Testing	The /api/analyze endpoint tested end-to-end with sample contracts of varying formats and lengths.
Engine Comparison Testing	Clauses tested with keyword detection disabled and semantic detection disabled independently, to confirm graceful degradation.
Sample Contract Testing	Fair, moderate, predatory, deceptive, and subtle sample contracts (public/samples/) used to validate that the risk level scales appropriately with contract quality.
UI Testing	Manual verification that highlighted clauses in the ContractViewer align with the correct text positions returned by the API.
PDF / Email Testing	Manual verification that generated PDF reports are correctly formatted and that email delivery, with and without a PDF attachment, succeeds.
Failure-Mode Testing	Semantic services (Hugging Face / ChromaDB) deliberately disabled to confirm the pipeline falls back to keyword-only detection without crashing.

Table 8.1: Testing Strategy Summary

The sample contracts provided in public/samples/, labelled fair, moderate, predatory, deceptive, and subtle, were particularly useful for regression testing: after any change to the scoring model or pattern database, all five samples were re-analyzed to confirm that the resulting risk levels remained correctly ordered — that is, that the predatory and deceptive samples continued to score meaningfully higher than the fair sample, and that the subtle sample, designed to contain paraphrased rather than keyword-obvious risk, was still caught primarily by the semantic engine rather than the keyword engine.

Chapter 9: Results and Discussion

Testing against the sample contract set demonstrated that the dual-engine approach materially increases detection coverage relative to either engine alone. Clauses containing well-known risky legal phrasing were reliably caught by the keyword engine, which also had the benefit of directly citing the relevant Indian Contract Act section, giving the user a concrete legal reference. Clauses that expressed similar risk through unusual or paraphrased wording — for example, indirect language around liability or termination — were frequently missed by the keyword engine but caught by the semantic engine, confirming the value of embedding-based similarity search as a complementary detection layer.

The fairness deviation module added a further, contract-wide dimension of insight beyond clause-level flags: even contracts with relatively few individually risky clauses could still be identified as unfavourable overall if their payment, termination, or liability terms deviated significantly from the fair-contract baseline. This distinction between clause-level risk and contract-level fairness was found to be an important part of giving users a complete picture, since a contract can be technically free of obviously risky clauses while still being structurally unbalanced.

The context-aware scoring model produced risk scores that scaled sensibly with the sample set — the fair sample contract consistently scored in the low-risk range, while the predatory and deceptive samples scored in the high-risk range, with the moderate and subtle samples falling in between, which is the expected ordering for a system intended to reliably distinguish contract quality.

From a system-resilience standpoint, deliberately disabling the semantic infrastructure confirmed that the pipeline continues to operate using keyword detection alone, satisfying the non-functional reliability requirement identified in Chapter 3. The role-based explanation feature was found, in informal review, to make the flagged clauses considerably more approachable for non-lawyer users, since the same clause could be read either from a freelancer's or a company's point of view depending on which side of the contract the user represented.

9.1 Limitations Observed

- The rule-based engine's coverage is only as good as the seeded clause_patterns and act_sections data, so gaps in that dataset translate directly into missed keyword-based detections.
- The semantic engine's threshold filtering involves a trade-off between recall and false positives; overly permissive thresholds surface benign clauses as risky, while overly strict thresholds miss genuine paraphrased risk.
- The current system supports English-language contracts only, which limits its applicability to the many contracts executed in regional Indian languages.
- OCR-based extraction from scanned images is more error-prone than extraction from native PDF or DOCX text, which can occasionally affect clause parsing accuracy for scanned documents.

Chapter 10: Ethical Considerations

Because AndhaKanoon processes sensitive legal documents that may contain personal, financial, or commercially confidential information, data privacy and confidentiality were treated as first-class design concerns rather than an afterthought. Uploaded documents are processed for the purpose of generating the requested analysis and are not retained beyond what is necessary for that analysis and any explicit save action taken by the signed-in user.

The system is also explicitly positioned as a decision-support tool rather than a replacement for professional legal advice. Every generated report carries a disclaimer clarifying that the analysis is AI-generated, may not capture every nuance of a specific legal situation, and should not be treated as a substitute for consultation with a qualified lawyer, particularly for high-value or high-stakes agreements.

Transparency was prioritized in the design of the detection pipeline itself: every flagged clause retains a record of which engine detected it (keyword, semantic, or both) and, where applicable, the specific Indian Contract Act section it relates to, so that a user or reviewer can trace the basis for a given risk flag rather than treating the system as an unexplainable black box.

Chapter 11: Conclusion and Future Scope

11.1 Conclusion

AndhaKanoon demonstrates that combining a deterministic, statute-grounded rule engine with a transformer-based semantic search engine produces a legal contract analysis system that is both explainable and resilient to the paraphrasing that purely keyword-based tools tend to miss. By layering fairness-deviation checking and context-aware, explainable scoring on top of clause-level detection, and by generating role-based plain-language explanations, the system moves beyond simple clause extraction toward a genuinely actionable analysis that a non-lawyer user can understand and act on. The modular architecture — with a clear separation between the Next.js application layer, the SQLite-backed rule engine, the Hugging Face and ChromaDB-backed semantic engine, and the PostgreSQL-backed persistence layer — also makes the system practical to extend, since each module can be improved or replaced independently of the others.

11.2 Future Scope

- Dataset Expansion: broaden the clause_patterns, act_sections, and fair_contract_baseline datasets, and fine-tune the semantic model on a larger, more diverse corpus of Indian legal contracts to improve detection accuracy.
- Multilingual Support: extend text extraction, clause parsing, and risk detection to contracts written in Hindi and other regional Indian languages.
- Voice / Chat Interface: incorporate a conversational AI assistant that allows a user to ask follow-up questions about a specific clause or the overall analysis in natural language.
- Comparative Analysis: extend the existing contract-comparison page into a full side-by-side risk comparison across two or more contract versions or offers.
- Negotiation Suggestions: generate concrete, ready-to-use alternative clause language that a user could propose to a counterparty to reduce identified risk.
- Mobile Application: package the analysis pipeline behind a dedicated mobile client for on-the-go contract review.

References

[1] I. Chalkidis et al., "LEGAL-BERT: The Muppets Straight Out of Law School," arXiv preprint arXiv:2010.02559, 2020.

[2] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," Proceedings of EMNLP System Demonstrations, pp. 38–45, 2020.

[3] "AI-Powered Legal Research, Contract Analysis, and Decision-Making," ResearchGate, August 2025.

[4] J. R. Ashley, Artificial Intelligence and Legal Analytics: New Tools for Law Practice in the Digital Age, Cambridge University Press, 2017.

[5] Kaggle, "Legal Case Reports and Contract Datasets," available at www.kaggle.com/datasets, accessed 2025.

[6] Hugging Face, "Transformers Documentation," available at huggingface.co/docs, accessed 2025.

[7] Chroma, "ChromaDB Documentation," available at docs.trychroma.com, accessed 2025.

[8] Vercel, "Next.js Documentation," available at nextjs.org/docs, accessed 2025.

[9] Prisma, "Prisma ORM Documentation," available at www.prisma.io/docs, accessed 2025.

[10] The Indian Contract Act, 1872, Government of India.

[11] Clerk, "Clerk Authentication Documentation," available at clerk.com/docs, accessed 2025.